Adversarial Distillation of Generative Models: A Unified View via Stochastic Interpolants

Author: Vladislav Gromadskii

github.com/WladGrm/si-adistill

1 Introduction

Generative models are now everywhere: they synthesize images [9], generate videos [10], fold proteins [11], and design molecular graphs [12]. Most rely on iterative refinement – diffusion, flow matching, or bridge matching – which produces remarkable quality, but demands tens to hundreds of network evaluations per sample. This might be too slow for real-time use.

The community has responded with a growing catalogue of *adversarial distillation* algorithms: SiD [3] for diffusion, Flow Generator Matching [4] for flow matching, Inverse Bridge Matching Distillation [5] for bridge matching, and so on. Each derives a seemingly distinct objective. At first glance these methods look different – but *do* they?

Key Observation

Diffusion, flow matching, and bridge matching are all instances of a single object: the **Stochastic Interpolant** [1]. I will derive the general adversarial distillation loss via the *Interpolant Generalisation Gap*. It can be written once and for all in terms of $(\alpha_t, \beta_t, \gamma_t)$ and the coupling π. To distil a new model family, one simply **plugs in** the appropriate schedule.

I validate this perspective by distilling stochastic interpolant model on toy scale Fashion-MNIST and then scale it to SiT [6] on ImageNet.

2 Method

One Framework to Rule Them All. A stochastic interpolant (SI) [1] between data $x_0 \sim p_{\text{data}}$ and a reference $x_1 \sim p_1$ is

$$x_t \;=\; \alpha_t\, x_0 \;+\; \beta_t\, x_1 \;+\; \gamma_t\, z, \qquad z \sim \mathcal{N}(0, I), \tag{1}$$

with boundary conditions $\alpha_0 = 1,\ \beta_0 = 0,\ \gamma_0 = \gamma_1 = 0,\ \alpha_1 = 0,\ \beta_1 = 1$, so that x_0 is pure data and x_1 is pure reference. The three coefficients have clear roles: α_t fades out the data, β_t fades in the reference, and γ_t injects additional noise that shapes the stochastic bridge. When $\gamma_t \equiv 0$ we get a deterministic interpolant (ODE); when $\gamma_t > 0$ we get an SDE with tunable diffusivity.
The key insight: *every popular generative framework is just a particular choice of* $(\alpha_t, \beta_t, \gamma_t)$ *and* p_1:

- **Flow matching** [2]: $\alpha_t = 1-t,\ \beta_t = t,\ \gamma_t = 0,\ p_1 = \mathcal{N}(0, I)$.
- **Diffusion (VP/DDPM)**: $\alpha_t = \sqrt{\bar{\alpha}_t},\ \beta_t = \sqrt{1-\bar{\alpha}_t},\ \gamma_t = 0,\ p_1 = \mathcal{N}(0, I)$. This is a one-sided interpolant [1] where the VP schedule satisfies $\alpha_t^2 + \beta_t^2 = 1$.
- **Bridge matching** [5]: $\alpha_t = 1-t,\ \beta_t = t,\ \gamma_t = \sqrt{t(1-t)}$, and p_1 can be *any* distribution (degraded images, another dataset, etc.)[1]

In all cases, a *teacher* ν^* is trained by velocity matching against $\dot{I}_t = \dot{\alpha}_t\, x_0 + \dot{\beta}_t\, x_1 + \dot{\gamma}_t\, z$:

$$\mathcal{L}_\pi(\nu) \;=\; \mathbb{E}\Big[\big\|\nu(x_t, t) \;-\; \dot{I}_t\big\|^2\Big]. \tag{2}$$

[1] Unlike flow matching or diffusion, which transport between *distributions*, bridge matching learns a *pinned* process: the trajectory is conditioned on both endpoints (x_0, x_1) drawn from a coupling $\pi(x_0, x_1)$. In practice the velocity network takes x_0 as an additional input, $\nu(x_t, t, x_0)$, so that at inference a concrete x_0 determines which bridge to generate along.

Velocity–Score Link for Diffusion. Diffusion needs score. Flow matching needs velocity. They are explicitly connected! SiT paper [6] establishes a direct connection between the velocity field ν and the score function $s(x_t, t) = \nabla_{x_t} \log p_t(x_t)$:

$$\nu(x_t, t) \;=\; \frac{\dot{\alpha}_t}{\alpha_t}\, x_t \;-\; \Big(\dot{\beta}_t - \frac{\dot{\alpha}_t\, \beta_t}{\alpha_t}\Big)\, \beta_t\, s(x_t, t). \tag{3}$$

This means that any pre-trained diffusion score model can be converted into a velocity field for our framework, and vice versa.

The Interpolant Generalisation Gap. A student generator G_θ maps noise to data in one shot, inducing a joint π_θ defined by $x_0 \sim p_{\text{data}},\ x_1' = G_\theta(x_0, z),\ z \sim \mathcal{N}(0, I)$. We measure how far this joint is from the teacher's via the gap:

$$\mathcal{G}(\theta) \;=\; \mathcal{L}_{\pi_\theta}(\nu^*) \;-\; \min_{\nu_\psi} \mathcal{L}_{\pi_\theta}(\nu_\psi). \tag{4}$$

Since ν^* was trained on real data, it incurs a high loss on the student's distribution; a *fake model* ν_ψ, adapting to π_θ, does better. Minimising $\mathcal{G}$ over θ while maximising over ψ gives the adversarial game:

$$\min_{G_\theta}\ \max_{\nu_\psi}\ \underbrace{\Big[\mathcal{L}_{\pi_\theta}(\nu^*) - \mathcal{L}_{\pi_\theta}(\nu_\psi)\Big]}_{\mathcal{G}(\theta)}. \tag{5}$$

Expanding via $\|a-c\|^2 - \|b-c\|^2 = (\|a\|^2 - \|b\|^2) - 2\langle a-b, c\rangle$:

$$\mathcal{G}(\theta) \;=\; \underbrace{\mathbb{E}\big[\|\nu^*\|^2 - \|\nu_\psi\|^2\big]}_{\text{norm terms (unstable)}} \;-\; 2\, \underbrace{\mathbb{E}\big[\langle \nu^* - \nu_\psi,\ \dot{I}_t' \rangle\big]}_{\text{cross term (stable)}}. \tag{6}$$

The norm terms involve second-order Jacobian products and are dropped in practice, leaving the *cross-term loss*:

$$\mathcal{L}_{\text{cross}}(\theta, \psi) \;=\; -2\, \mathbb{E}\big[\langle \nu^*(x_t', t) - \nu_\psi(x_t', t),\ \ \dot{I}_t' \rangle\big]. \tag{7}$$

This has a clean interpretation: the loss measures how the teacher–fake velocity mismatch *projects* onto the student's generation direction. If the student generates well, the two models agree, and the loss vanishes.

3 Experiments

I pretrained multi-step teacher Unet model on Fashion-MNIST and SiT-S/2 on ImageNet and distilled them into 1-step generator G_θ with the cross-term loss (7). Both G_θ and ν_ψ are initialised from the teacher weights. I used exponential moving average (EMA) on both the fake and student models for training stability.

(a) Fashion-MNIST. Teacher (50 steps, left) vs. student (1 step, right).

(b) ImageNet 256×256 SiT-S/2. Teacher, 250 steps, FID = 61.5 (left pair) vs. student, 1 step, FID = 59.8 (right pair).

Figure 1: Teacher vs. distilled 1-step student.

4 Discussion

In this work I derived a general adversarial loss (5) for the stochastic interpolant framework, which absorbs most of the ODE/SDE-based generative families into Eq. (1) and experimentally proved its validity and scalability. Beyond that, it opens new research directions, such as theoretically and practically expanding on other emerging classes of generative models, such as discrete diffusion [7] and discrete flow matching [8].

5 References

[1] Albergo, M. S., Boffi, N. M. and Vanden-Eijnden, E. *Stochastic Interpolants: A Unifying Framework for Flows and Diffusions.* arXiv:2303.08797, 2023.

[2] Lipman, Y. et al. *Flow Matching for Generative Modeling.* ICLR, 2023.

[3] Zhou, M. et al. *Score identity Distillation: Exponentially Fast Distillation of Pretrained Diffusion Models for One-Step Generation.* ICML, 2024.

[4] Huang, Z., Geng, Z., Luo, W. and Qi, G.-j. *Flow Generator Matching.* arXiv:2410.19310, 2024.

[5] Gushchin, N. et al. *Inverse Bridge Matching Distillation.* arXiv:2502.01362, 2025.

[6] Ma, N. et al. *SiT: Exploring Flow and Diffusion-based Generative Models with Scalable Interpolant Transformers.* ECCV, 2024.

[7] Sahoo, S. S. et al. *Simple and Effective Masked Diffusion Language Models.* NeurIPS, 2024.

[8] Gat, I. et al. *Discrete Flow Matching.* arXiv:2407.15595, 2024.

[9] Rombach, R. et al. *High-Resolution Image Synthesis with Latent Diffusion Models.* CVPR, 2022.

[10] Ho, J. et al. *Video Diffusion Models.* NeurIPS, 2022.

[11] Watson, J. L. et al. *De Novo Design of Protein Structure and Function with RFdiffusion.* Nature, 2023.

[12] Vignac, C. et al. *DiGress: Discrete Denoising Diffusion for Graph Generation.* ICLR, 2023.